



TheDBAdmin Practitioner Series

PostgreSQL Architecture

How the Engine Works, From the
Process Model to the 8 kB Page

A Practitioner's Guide to the Internals · PostgreSQL 18

www.thedbadmin.com

PostgreSQL Architecture

How the Engine Works, From the Process Model to the 8 kB Page

© 2026 TheDBAdmin. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of TheDBAdmin, except for brief quotations in reviews and certain other non-commercial uses permitted by copyright law.

This book is part of the TheDBAdmin Practitioner Series. The procedures herein were validated against vendor documentation current as of June 2026. Software behavior changes over time; always confirm against the current official documentation for your versions before applying any procedure to a production system.

NOTE — About the inspection commands

The SQL and shell commands in this book are read-only diagnostics, written for PostgreSQL 18 and verified against the official documentation. Run them on a scratch instance first; output such as object IDs, file paths, and transaction IDs will differ on your system.

Contact

Web: www.thedbadmin.com

Email: admin@thedbadmin.com

Phone: +91 6232090055

First Edition · 2026

Table of Contents

Preface — How to Read This Book.....	1
The engine at a glance.....	1
Chapter 1 — The Process Model.....	3
The problem: who runs your query?.....	3
The core idea: one process per connection.....	3
The architecture: postmaster, backends, background processes.....	3
The workflow: a connection's life.....	3
Worked example: see the processes.....	3
Best practices, trade-offs, and mistakes.....	4
Advanced and comparable tools.....	4
Chapter 2 — Memory Architecture.....	4
The problem: where does the RAM go?.....	4
The core idea: shared memory vs. local memory.....	4
The workflow and a worked example.....	5
Best practices, trade-offs, and mistakes.....	5
Chapter 3 — On-Disk Layout.....	6
The problem: what is actually in the data directory?.....	6
The core idea and architecture.....	6
Worked example: find a table's files.....	7
Best practices, mistakes, and advanced notes.....	7
Chapter 4 — The Page and the Tuple.....	7
The problem: how is a row physically stored?.....	7
The core idea and architecture.....	7
Worked example: look inside a page.....	8
Best practices, trade-offs, mistakes, and advanced notes.....	8
Chapter 5 — MVCC: Concurrency Without Read Locks.....	9
The problem: readers and writers colliding.....	9
The core idea and architecture.....	9
Worked example: watch the versions.....	9
Best practices, trade-offs, mistakes, and advanced notes.....	10
Chapter 6 — WAL and the Write Path.....	10
The problem: durability without crippling writes.....	10
The core idea and architecture.....	10
Worked example: observe WAL and checkpoints.....	11
Best practices, trade-offs, mistakes, and advanced notes.....	11
Chapter 7 — The Buffer Manager and Asynchronous I/O.....	12
The problem: disk is slow, RAM is small.....	12
The core idea and architecture.....	12
Worked example: inspect the pool and I/O settings.....	13
Best practices, trade-offs, mistakes, and advanced notes.....	13
Chapter 8 — The Life of a Query.....	14
The problem: from SQL text to rows.....	14
The core idea and architecture.....	14
Worked example: read the plan.....	14

Best practices, trade-offs, mistakes, and advanced notes.....	15
Chapter 9 — VACUUM, Autovacuum, and the Cost of MVCC.....	15
The problem: dead rows and the doomsday clock.....	15
The core idea and architecture.....	15
Worked example: watch autovacuum work.....	16
Best practices, trade-offs, mistakes, and advanced notes.....	16
Chapter 10 — A Guided Tour: One Write, End to End.....	17
Summary — The Whole Engine in One View.....	18
References.....	18
About TheDBAdmin.....	19

Preface — How to Read This Book

Most database engineers meet PostgreSQL as a black box: a connection string goes in, rows come out. That model is enough until the day a query plan turns pathological, a table bloats to ten times its data size, or a server runs out of memory under load. On that day the people who understand **how the engine actually works** fix the problem in minutes; everyone else guesses. This book is about becoming the first kind of engineer.

Every chapter follows the same nine-step path, so you always meet a concept the same way: (1) the problem it solves, (2) the core idea, (3) the architecture, (4) the workflow, (5) a worked example you can run, (6) best practices, (7) trade-offs, (8) common mistakes, and (9) advanced extensions and how PostgreSQL compares to other engines. We teach from first principles — the physics of disks and processes — before reaching for tunables.

NOTE — Versions and verification

This edition targets **PostgreSQL 18**, the current major release, which introduced an asynchronous I/O subsystem. Every command and figure was checked against the official PostgreSQL 18 documentation. Where a default value matters, the version-specific number is stated explicitly.

The engine at a glance

Figure 1 is the map for the whole book. PostgreSQL is a **multi-process** system: a supervisor (the postmaster) forks one operating-system process per client connection, and a handful of background processes do the housekeeping. They all communicate through a single region of shared memory, and everything durable eventually lands in two places on disk — the data files and the write-ahead log.

Figure 1. PostgreSQL's Process & Memory Architecture

One supervisor, one process per connection, one shared memory region

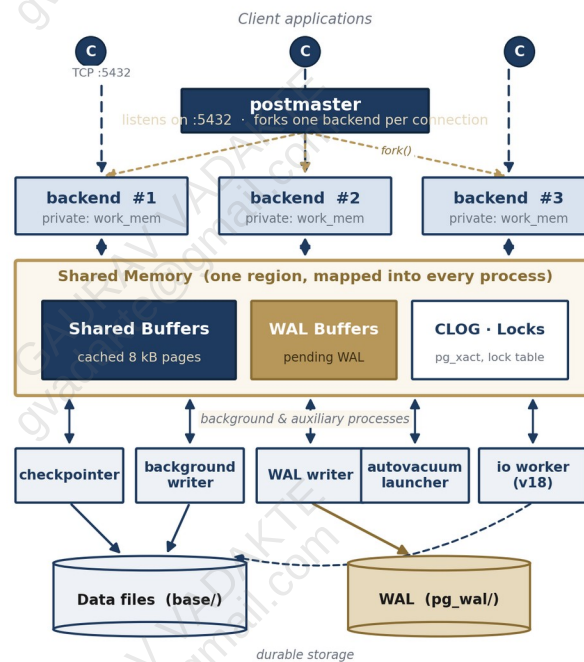


Figure 1. The process and memory architecture that the rest of this book unpacks, layer by layer.

Hold this picture in mind. Storage (Part II) is what lives in the data files; the buffer manager is how those files reach the shared buffers; WAL is how changes become durable; and query execution and VACUUM (Part III) are what the backend and the autovacuum workers actually do all day.

PART I

The Mental Model — Processes and Memory

Chapter 1 — The Process Model

The problem: who runs your query?

When a client connects, something on the server has to authenticate it, parse its SQL, read and write data, and respect what every other client is doing at the same time. The first architectural decision any database makes is *what that “something” is* — a thread, a process, or a pool of workers. PostgreSQL's answer shapes everything else, including why connection pooling is not optional at scale.

The core idea: one process per connection

PostgreSQL uses a **process-per-connection** model. A supervisor process called the **postmaster** listens on port 5432, and for every accepted connection it `fork()`s a dedicated child — a **backend**. That backend serves exactly one client for the life of the session and then exits. Because each backend is a full OS process, the kernel gives PostgreSQL process isolation and scheduling for free: a crash in one backend cannot corrupt another's memory.

The architecture: postmaster, backends, background processes

Refer back to Figure 1. Around the backends sit the **auxiliary processes**, each with one job: the **checkpointer** and **background writer** flush dirty pages to disk; the **WAL writer** flushes the write-ahead log; the **autovacuum launcher** spawns workers that clean up dead rows; the **archiver** and **logical replication launcher** support backups and replication; and, new in version 18, **I/O workers** perform reads asynchronously on behalf of any backend.

The workflow: a connection's life

1. The postmaster accepts a TCP (or Unix-socket) connection on 5432.
2. It forks a backend and hands off the socket; authentication happens in the child.
3. The backend attaches to shared memory and serves queries until the client disconnects, then exits.

Worked example: see the processes

From the shell and from SQL you can watch the model directly. The backend types reported by the server are the same processes drawn in Figure 1:

```
# Every PostgreSQL process on the host
ps -e -o pid,cmd | grep '^ *[0-9]* postgres'

-- From inside psql: who is connected and what is each process?
```

```
SELECT pid, backend_type, state, query
FROM pg_stat_activity
ORDER BY backend_type;
```

On version 18 you will see rows whose `backend_type` is `client backend`, alongside `checkpointer`, `background writer`, `walwriter`, `autovacuum launcher`, and `io worker`.

Best practices, trade-offs, and mistakes

- **Pool your connections.** Forking a process and warming its caches is expensive. A pooler such as PgBouncer lets thousands of clients share a few hundred backends. This is the single most important operational consequence of the process model.
- **Set `max_connections` honestly.** Each backend reserves memory for its private work areas; a few hundred is healthy, several thousand is usually a pooling failure in disguise.
- **The trade-off:** processes cost more to create than threads and use more memory per connection, but they give robust isolation and let PostgreSQL lean on the mature, battle-tested OS process scheduler.

WARNING — The classic mistake

Pointing an application server with its own large connection pool directly at PostgreSQL, multiplied across many app instances, until `max_connections` is exhausted and new sessions are refused. The fix is a pooler, not a higher connection limit.

Advanced and comparable tools

Backends and background workers can be extended: the *background worker* API lets extensions (and parallel query) run their own server processes. By contrast, MySQL/InnoDB uses a **thread-per-connection** model, and Oracle offers both dedicated-server (process-per-connection, like PostgreSQL) and shared-server modes. The trade-offs are the eternal ones: threads share an address space and are cheaper but less isolated; processes are heavier but safer.

Chapter 2 — Memory Architecture

The problem: where does the RAM go?

A database server with 64 GB of RAM can still run out of memory and be killed by the kernel — or, just as bad, leave most of that RAM unused. To size a server correctly you must know which memory is **shared** once across the whole instance and which is **private** and multiplied by every backend and every operation.

The core idea: shared memory vs. local memory

PostgreSQL allocates one large **shared memory** region at startup, mapped into every process. Its biggest tenant is the **shared buffers** — the cache of 8 kB data pages — followed by the **WAL buffers**, the commit-log (CLOG) buffers, and the lock tables. Separately, each backend has **local memory** for its own work: `work_mem` for sorts and hashes, `maintenance_work_mem` for VACUUM and index builds, and `temp_buffers` for temporary tables.

The workflow and a worked example

Inspect the live configuration rather than trusting documentation defaults, which builds and packagers sometimes override:

```
SELECT name, setting, unit, context
FROM pg_settings
WHERE name IN ('shared_buffers', 'wal_buffers', 'work_mem',
               'maintenance_work_mem', 'temp_buffers', 'effective_cache_size');
```

Out of the box, `shared_buffers` defaults to a conservative 128 MB, `work_mem` to 4 MB, and `maintenance_work_mem` to 64 MB — values meant to start anywhere, not to perform well on a real server.

Best practices, trade-offs, and mistakes

- **Shared buffers $\approx$ 25% of RAM** is the usual starting point. PostgreSQL deliberately relies on the operating-system page cache for the rest, so you are not trying to cache the whole database in shared buffers.
- **work_mem is per operation, not per query.** A single complex query with several sorts and hash joins, run by many connections at once, multiplies `work_mem` many times over. Size it for the workload, not for one query in isolation.
- **effective_cache_size is a hint, not an allocation.** It tells the planner how much cache (shared buffers plus OS cache) it can expect, influencing index-vs-scan choices, but allocates nothing.

WARNING — The classic mistake

Setting `work_mem` to a large value globally to speed up one report, then watching the server be OOM-killed under concurrency. Raise it per-session or per-role for the queries that need it instead.

If you know Oracle, the mapping is familiar: shared buffers and the rest of PostgreSQL's shared memory play the role of the **SGA**, while each backend's local memory is its **PGA**. The crucial difference is PostgreSQL's deliberate reliance on the OS page cache, which is why double-counting it is a real tuning hazard — the subject we return to in Chapter 7.

PART II

Storage, Concurrency, and Durability

Chapter 3 — On-Disk Layout

The problem: what is actually in the data directory?

When a disk fills up, a backup must be verified, or a table mysteriously occupies far more space than its rows, you need to know what the files under the data directory mean. PostgreSQL's on-disk layout is logical and inspectable once you learn its three levels: the cluster, the database, and the relation.

The core idea and architecture

A running server manages one **data directory** (referred to by the environment variable `$PGDATA`), which holds one **cluster** — a collection of databases. Inside it, `base/` holds one subdirectory per database, `global/` holds cluster-wide catalogs, `pg_wal/` holds the write-ahead log, and `pg_xact/` holds commit status. Figure 2 traces the path from `$PGDATA` down to the individual files of one table.

Figure 2. On-Disk Layout of a Database Cluster

From `$PGDATA` down to the 1 GB segment files of a single table

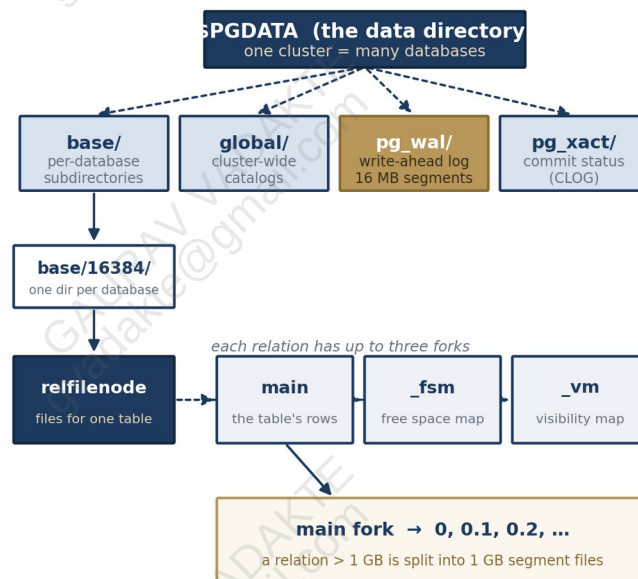


Figure 2. The data directory, from the cluster down to the 1 GB segment files of a single relation.

Each table or index is a **relation**, stored as one or more **forks**: the main fork holds the rows, the `_fsm` fork (free space map) records where free space is, and the `_vm` fork (visibility map) marks pages where every row is visible to all transactions. A relation larger than **1 GB** is split into numbered 1 GB segment files.

Worked example: find a table's files

```
-- The path (relative to $PGDATA) of a table's main fork
SELECT pg_relation_filepath('myschema.mytable');
-- e.g. base/16ptable_oid

-- Size on disk, including indexes and TOAST
SELECT pg_size_pretty(pg_total_relation_size('myschema.mytable'));
```

Best practices, mistakes, and advanced notes

- **Never touch the files directly.** The server owns them. Use SQL, base backups, and tablespaces (a tablespace simply points a relation's files at a different filesystem).
- **OID is not always relfilenode.** A table's on-disk filename is its relfilenode, which usually equals its OID but changes after operations like `VACUUM FULL` or `TRUNCATE`. Resolve paths with `pg_relation_filepath`, never by assuming the OID.
- **TOAST handles the oversized.** Values too large to fit comfortably in a page (roughly 2 kB) are compressed and/or moved to an associated TOAST table, transparently. A “small” table with huge text columns hides most of its bytes there.

The hierarchy echoes Oracle's tablespaces → segments → extents → blocks, but PostgreSQL's is simpler: one file (in 1 GB chunks) per fork per relation, with the filesystem doing the block management Oracle does itself.

Chapter 4 — The Page and the Tuple

The problem: how is a row physically stored?

Indexes, MVCC, bloat, and even the best column order all follow from one structure: the **8 kB page**. Understanding the page is what turns vague advice (“avoid wide rows”, “watch your fillfactor”) into something you can reason about.

The core idea and architecture

Every table and index is an array of fixed-size **pages** (also called blocks), 8 kB by default. As Figure 3 shows, a page has a 24-byte header, then an array of 4-byte **line pointers** that grows downward, while the **tuples** (rows) themselves are added upward from the bottom. The gap between them is the page's free space.

Figure 3. Inside an 8 kB Heap Page

Line pointers grow down, tuples grow up; they meet at the free space

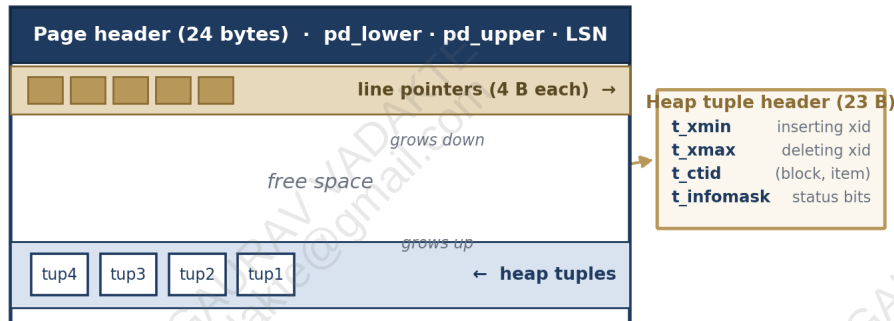


Figure 3. A heap page: line pointers grow down, tuples grow up, and each tuple carries the MVCC header.

Each heap tuple begins with a 23-byte header. Four of its fields drive the entire concurrency system: `t_xmin` (the transaction that inserted the row), `t_xmax` (the transaction that deleted or superseded it), `t_ctid` (the physical address — block number and item number — of this or the next version), and `t_infomask` (status bits).

Worked example: look inside a page

The `pageinspect` extension lets you read raw page contents — invaluable for teaching and forensics:

```
CREATE EXTENSION IF NOT EXISTS pageinspect;

-- Header of block 0
SELECT * FROM page_header(get_raw_page('mytable', 0));

-- The line pointers and tuple metadata on block 0
SELECT lp, t_xmin, t_xmax, t_ctid
FROM heap_page_items(get_raw_page('mytable', 0));
```

Best practices, trade-offs, mistakes, and advanced notes

- **Mind alignment.** Columns are padded to alignment boundaries. Ordering columns from widest fixed-width types down to narrow ones can shrink rows and fit more per page — free density with no schema change in meaning.
- **Use `fillfactor` for update-heavy tables.** Leaving free space on each page (a lower `fillfactor`) lets updates place the new row version on the same page — a HOT update, covered in Chapter 5.
- **The 8 kB size is a compile-time constant.** It is a deliberate balance: large enough to amortize I/O, small enough to limit write amplification. Changing it requires recompiling and reinitializing, so in practice it is fixed.

NOTE — Why this matters for bloat

Because tuples are packed into fixed pages and old versions linger (Chapter 5), a table can hold far more pages than its live rows justify. The page model is the reason bloat exists — and the reason VACUUM can fix it.

Chapter 5 — MVCC: Concurrency Without Read Locks

The problem: readers and writers colliding

If a long report is scanning a table while transactions update it, a naive engine must choose: block the writers, block the reader, or show inconsistent data. PostgreSQL refuses all three. Its answer is **Multiversion Concurrency Control (MVCC)**, and it is the defining feature of the storage engine.

The core idea and architecture

PostgreSQL **never overwrites a row in place**. An UPDATE writes a *new* version of the row and marks the old one as expired by stamping its `t_xmax`. As Figure 4 shows, both versions then coexist on the heap. Each transaction sees the database through a **snapshot**: it sees a row version only if that version's `t_xmin` committed before the snapshot was taken and its `t_xmax` has not. Readers therefore never block writers, and writers never block readers.

Figure 4. MVCC — an UPDATE Writes a New Row Version

PostgreSQL never overwrites a row in place; readers never block writers

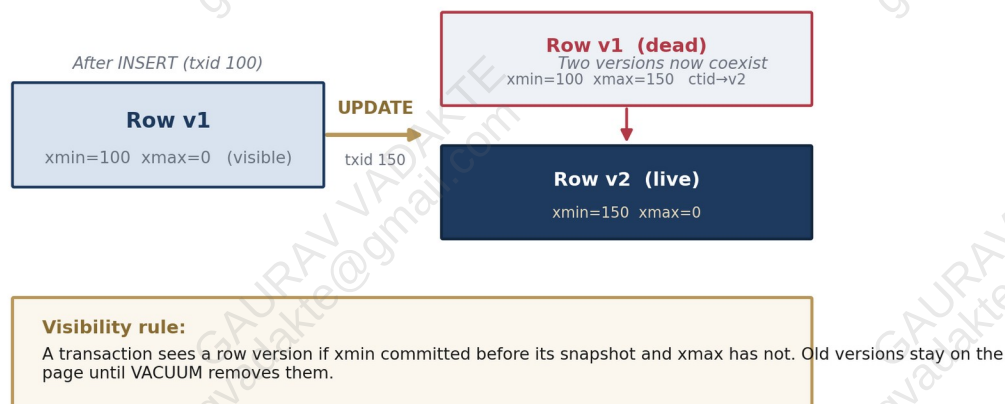


Figure 4. An UPDATE leaves the old version in place and writes a new one; visibility is decided per transaction.

Worked example: watch the versions

```
-- xmin/xmax/ctid are hidden system columns on every table
SELECT ctid, xmin, xmax, * FROM accounts WHERE id = 1;

UPDATE accounts SET balance = balance + 100 WHERE id = 1;
```

```
-- Same row, new ctid and new xmin; the old version still
-- exists on disk until VACUUM removes it.
SELECT ctid, xmin, xmax, * FROM accounts WHERE id = 1;
```

Best practices, trade-offs, mistakes, and advanced notes

- **Keep transactions short.** The oldest running transaction sets the “xmin horizon”: no dead row newer than it can be cleaned up. A single forgotten transaction left open for hours pins bloat across the entire database.
- **The trade-off is bloat.** MVCC buys lock-free reads at the cost of dead row versions that must be reclaimed. That bill is paid by VACUUM (Chapter 9) — MVCC and VACUUM are two halves of one design.
- **HOT updates are the optimization.** If an update changes no indexed column and the new version fits on the same page, PostgreSQL performs a Heap-Only Tuple update: it chains the versions by ctid and avoids touching the indexes entirely.

WARNING — The classic mistake

A session left idle in transaction — often an application that forgot to commit. It holds the xmin horizon open, autovacuum cannot remove dead rows, and tables bloat silently. Monitor `pg_stat_activity` for it.

Oracle and MySQL/InnoDB also provide multiversion reads, but they keep old versions in a separate *undo* area and reconstruct prior images on demand. PostgreSQL keeps every version in the table itself — simpler and crash-friendly, at the price of needing VACUUM to tidy up.

Chapter 6 — WAL and the Write Path

The problem: durability without crippling writes

A committed transaction must survive a crash one millisecond later. The brute-force way — flush every changed data page to disk at commit — would scatter tiny random writes across the disk and be ruinously slow. PostgreSQL needs durability *and* speed, and it gets both with the write-ahead log.

The core idea and architecture

The rule of **Write-Ahead Logging (WAL)** is simple: describe every change in a compact log record and flush *that* to disk before the change is considered durable — before the data file itself is updated. As Figure 5 shows, a backend appends a record to the WAL buffers and modifies the page in shared buffers; at COMMIT only the WAL is flushed (fsynced) to `pg_wal/`. The dirty data page is written out lazily, much later, by the checkpoint.

Figure 5. The Write Path — WAL First, Data Files Later

Commit waits only for the small sequential WAL flush, not the data files

WAL is written and flushed BEFORE the data file changes — that is the **durability guarantee**

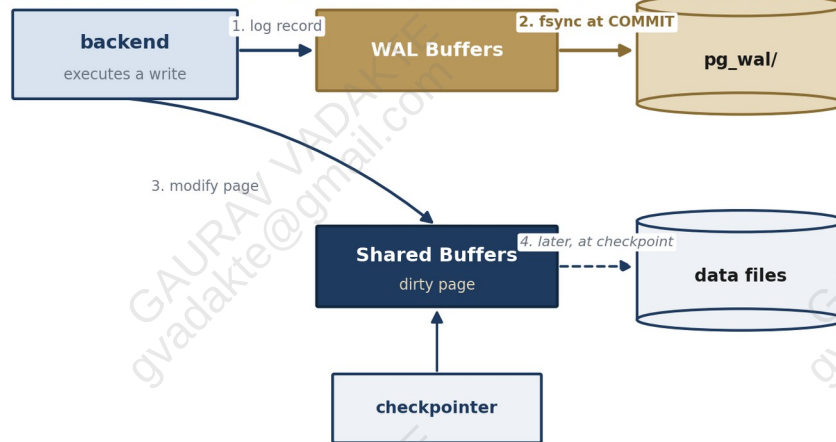


Figure 5. Commit waits only for a small, sequential WAL flush; data files are reconciled later at a checkpoint.

Because WAL is written sequentially, the commit-time flush is fast. After a crash, recovery **replays** the WAL forward from the last **checkpoint** — a known-good point where all dirty pages had been flushed — reconstructing every committed change. WAL segments are 16 MB files by default.

Worked example: observe WAL and checkpoints

```
-- Current write position in the WAL stream
SELECT pg_current_wal_lsn();

-- Force a checkpoint, then inspect control data from the shell
CHECKPOINT;
-- $ pg_controldata $PGDATA (shows latest checkpoint LSN, etc.)
```

Best practices, trade-offs, mistakes, and advanced notes

- **Tune checkpoints to spread the I/O.** Raise `max_wal_size` and keep `checkpoint_completion_target` near its 0.9 default so the checkpointer trickles writes out rather than flushing a flood at once.
- **Understand synchronous_commit.** Turning it off lets commits return before the WAL flush completes — dramatically faster, but a crash can lose the last few transactions. It is a durability dial, not a free lunch.
- **full_page_writes protects against torn pages.** The first change to a page after each checkpoint logs the whole page, guarding against partial 8 kB writes during a crash. Leave it on.

WARNING — The classic mistake

Setting `fsync = off` to “speed things up”. It removes the durability guarantee entirely: a

power loss can leave the cluster corrupt and unrecoverable. Never do this on data you care about.

WAL is also the foundation of PostgreSQL's replication and point-in-time recovery: a standby simply replays the primary's WAL stream. Oracle's redo log and InnoDB's redo log serve the same write-ahead purpose; the concept is universal because the physics of disks is universal.

Chapter 7 — The Buffer Manager and Asynchronous I/O

The problem: disk is slow, RAM is small

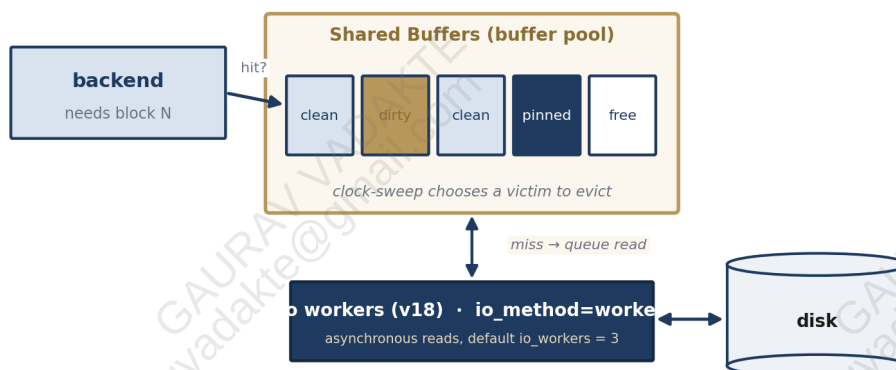
A database is usually far larger than memory, yet most queries must feel as if the data were in RAM. The buffer manager is the layer that decides which 8 kB pages live in memory, when they are written back, and — new in version 18 — how reads are issued so the CPU need not sit idle waiting for the disk.

The core idea and architecture

The **shared buffers** are a pool of 8 kB frames. When a backend needs a page, it looks for it in the pool; a **hit** is served from RAM, a **miss** must be read from disk into a free or evicted frame. Eviction uses a **clock-sweep** algorithm that approximates least-recently-used by decrementing a usage counter on each pass and reclaiming the first frame to reach zero. Dirty frames must be written out (by the checkpoint or background writer) before reuse.

Figure 6. The Buffer Manager & Asynchronous I/O (v18)

Backends read through the shared pool; v18 offloads misses to I/O workers



A page lives in the pool until evicted; dirty pages are flushed by the checkpoint or background writer first.

Figure 6. Reads pass through the shared pool; version 18 hands misses to asynchronous I/O workers.

Version 18 adds an **asynchronous I/O** subsystem controlled by `io_method`. The default, `worker`, dispatches read requests to a small pool of **I/O worker** processes (default `io_workers = 3`) so

backends can queue many reads and keep working while the disk catches up; `io_uring` uses the Linux kernel interface directly, and `sync` reproduces the older blocking behavior.

Worked example: inspect the pool and I/O settings

```
CREATE EXTENSION IF NOT EXISTS pg_buffercache;

-- How many buffers are dirty right now?
SELECT count(*) FILTER (WHERE isdirty) AS dirty, count(*) AS total
FROM pg_buffercache;

-- Version 18 asynchronous I/O configuration
SHOW io_method;           -- worker (default)
SHOW io_workers;          -- 3
SHOW effective_io_concurrency; -- 16 in v18
```

Best practices, trade-offs, mistakes, and advanced notes

- **Do not try to cache everything.** Beyond roughly 25% of RAM, larger shared buffers compete with the OS page cache and can hurt. PostgreSQL is designed to use both layers.
- **Raise `effective_io_concurrency` on SSDs.** Version 18 already defaults it to 16 (up from 1); fast NVMe storage can benefit from going higher, which lets prefetch and AIO issue more parallel reads.
- **The double-buffering trade-off.** A page can sit in both shared buffers and the OS cache. That overlap is usually acceptable, but it is why blindly enlarging shared buffers is not always a win.

NOTE — Why version 18 matters here

Before 18, a buffer miss blocked the backend until the read returned. With `io_method = worker`, sequential scans, bitmap heap scans, `VACUUM`, and `ANALYZE` can overlap I/O with computation — a structural speed-up for analytic and maintenance workloads.

PART III

Execution and Maintenance

Chapter 8 — The Life of a Query

The problem: from SQL text to rows

Two queries that return the same rows can differ in runtime by orders of magnitude, depending on the **plan** PostgreSQL chooses. To tune queries you must see the engine the way it sees itself: as a pipeline that turns text into a plan and a plan into rows.

The core idea and architecture

A query passes through four stages (Figure 7). The **parser** turns SQL text into a parse tree. The **analyzer and rewriter** resolve names against the catalog and apply rules, including view expansion. The **planner/optimizer** is the heart: it enumerates ways to execute the query — sequential scan or index scan, nested-loop or hash join, this join order or that — and uses **cost estimates** derived from table statistics to choose the cheapest. The **executor** then runs that plan, pulling tuples through a tree of nodes one at a time.

Figure 7. The Life of a Query

Four stages turn SQL text into rows



The executor pulls rows through a tree of nodes (sequential scan, index scan, join, sort) one tuple at a time.

Figure 7. Four stages turn SQL into rows; the planner's choices are driven by statistics.

Worked example: read the plan

```
-- Make sure statistics are current, then see the real plan
ANALYZE orders;
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE customer_id = 42 ORDER BY created_at;
```

Read the output inside-out: each node shows its estimated vs. actual rows and its buffer hits and reads. A large gap between estimated and actual rows is the single most useful signal that the planner is working from bad statistics.

Best practices, trade-offs, mistakes, and advanced notes

- **Keep statistics fresh.** The planner is only as good as the row-count and distribution estimates ANALYZE collects. Autovacuum runs ANALYZE automatically; do not disable it.
- **The cost model is an estimate.** Costs are heuristics over statistics, not measurements. Correlated columns, skewed data, and custom functions can mislead it; extended statistics and occasional query hints-by-rewrite are the remedies.
- **Parallelism and JIT are advanced gears.** Large scans can be split across parallel workers, and long analytic queries can be JIT-compiled. Both help big queries and add overhead to tiny ones.

WARNING — The classic mistake

Blaming the index when the real culprit is stale statistics. After a bulk load, run ANALYZE before judging query plans — the planner cannot choose well from numbers that describe an empty table.

Chapter 9 — VACUUM, Autovacuum, and the Cost of MVCC

The problem: dead rows and the doomsday clock

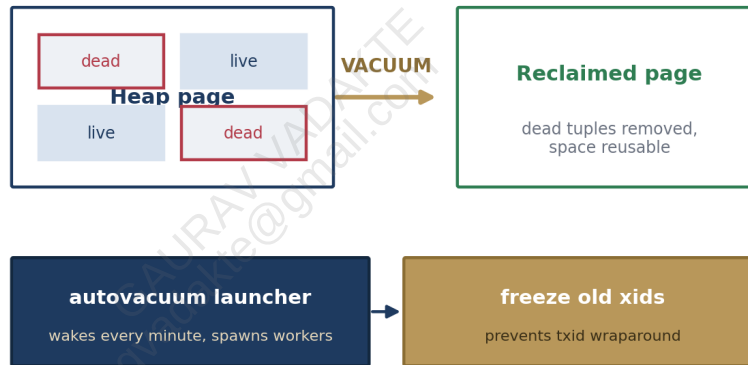
MVCC (Chapter 5) leaves dead row versions behind, and PostgreSQL's transaction IDs are a finite 32-bit counter. Left unmanaged, the first problem is **bloat** — tables and indexes swelling with dead tuples — and the second is **transaction ID wraparound**, which, if ever reached, would force the database into a protective shutdown. VACUUM exists to prevent both.

The core idea and architecture

VACUUM scans pages, removes dead tuples whose deleting transaction is older than every running snapshot, and makes that space reusable. It also **freezes** very old live rows — marking them as permanently visible — which is what holds the wraparound clock at bay. As Figure 8 shows, the **autovacuum launcher** wakes roughly once a minute and dispatches workers to tables that have accumulated enough changes.

Figure 8. VACUUM & Autovacuum

MVCC's bill comes due: dead tuples must be reclaimed and xids frozen



VACUUM also updates the visibility map (enabling index-only scans) and the free space map.

Figure 8. VACUUM reclaims dead tuples and freezes old transaction IDs; autovacuum does it automatically.

Autovacuum triggers on a threshold plus a scale factor: by default a table is vacuumed after roughly **50 + 20%** of its rows have changed, and an anti-wraparound vacuum is forced as a table's oldest unfrozen ID approaches `autovacuum_freeze_max_age` (default 200 million). VACUUM also maintains the visibility map, which enables **index-only scans**, and the free space map.

Worked example: watch autovacuum work

```
-- Dead tuples and last (auto)vacuum per table
SELECT relname, n_live_tup, n_dead_tup,
       last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;

-- See exactly what a manual vacuum does
VACUUM (VERBOSE, ANALYZE) orders;
```

Best practices, trade-offs, mistakes, and advanced notes

- **Tune autovacuum per table.** A large, hot table often needs a smaller scale factor so it is vacuumed more often in smaller, cheaper passes rather than rarely in painful ones.
- **Monitor wraparound proactively.** Track the oldest unfrozen transaction age per database (`datfrozenxid`); alert long before the `freeze-max-age` limit, never after.
- **The trade-off is I/O.** Aggressive vacuuming spends disk bandwidth now to avoid bloat and emergency anti-wraparound vacuums later. Version 18's asynchronous I/O makes those passes noticeably cheaper.

WARNING — The classic mistake

Disabling autovacuum to “reduce load” during a busy period. Dead tuples and transaction age keep climbing; the eventual forced anti-wraparound vacuum is far more disruptive than the steady background work it replaced. Tune it — never switch it off.

For tables already badly bloated, `VACUUM FULL` rewrites the table compactly but takes an exclusive lock; the `pg_repack` extension achieves a similar result with far less locking. Engines that use a separate undo area (Oracle, InnoDB) avoid in-table dead tuples but pay elsewhere — in undo management and the cost of reconstructing old versions.

Chapter 10 — A Guided Tour: One Write, End to End

To see the architecture as a whole, follow a single `UPDATE` through every layer of this book. Each step names the chapter where it was explained.

4. The **backend** serving your session (Ch. 1) receives the SQL and runs it through parse, plan, and execute (Ch. 8).
5. The executor needs the target page. The **buffer manager** (Ch. 7) finds it in shared buffers or, on a miss, has an I/O worker read it from the **data files** (Ch. 3).
6. The update writes a **new tuple version** on the page and stamps the old one's `t_xmax` (Ch. 4, Ch. 5), marking the buffer dirty.
7. A **WAL record** describing the change is appended to the WAL buffers; at `COMMIT` it is flushed to `pg_wal/` (Ch. 6). The transaction is now durable.
8. Later, the **checkpointer** flushes the dirty data page to disk (Ch. 6, Ch. 7). Later still, **autovacuum** reclaims the old row version and, in time, freezes the new one (Ch. 9).

BEST PRACTICE — Inspect your own server

The diagnostics below are read-only and safe on a non-production instance. Together they let you observe every layer this book described, live.

What you want to see	How to see it (PostgreSQL 18)
Processes and connections	<code>SELECT pid, backend_type, state FROM pg_stat_activity;</code>
Memory and I/O settings	<code>SELECT name, setting, unit FROM pg_settings WHERE name LIKE '%buffers%' OR name LIKE 'io_%';</code>
A table's files on disk	<code>SELECT pg_relation_filepath('t');</code>
Inside a page	<code>SELECT * FROM heap_page_items(get_raw_page('t',0)); (pageinspect)</code>
Row versions (MVCC)	<code>SELECT ctid, xmin, xmax, * FROM t;</code>
WAL position & checkpoint	<code>SELECT pg_current_wal_lsn(); then CHECKPOINT;</code>
Buffer pool contents	<code>SELECT count(*) FILTER (WHERE isdirty) FROM pg_buffercache;</code>
A query's plan	<code>EXPLAIN (ANALYZE, BUFFERS) <query>;</code>
Dead tuples & autovacuum	<code>SELECT relname, n_dead_tup, last_autovacuum FROM</code>

pg_stat_user_tables;

Summary — The Whole Engine in One View

PostgreSQL is a small number of cooperating ideas. A supervisor forks a process per connection; those processes share one memory region whose largest tenant caches 8 kB pages; rows live on those pages as immutable versions; durability comes from logging changes before applying them; and a background fleet of writers and vacuum workers keeps the whole thing flushed, compact, and crash-safe.

Layer	Component	Its one job
Processes	postmaster + backends	one process per connection (Ch. 1)
Memory	shared buffers + local work areas	cache pages; private sort/hash space (Ch. 2, 7)
Storage	\$PGDATA, forks, 8 kB pages	lay rows out on disk (Ch. 3, 4)
Concurrency	MVCC (xmin/xmax)	lock-free consistent reads (Ch. 5)
Durability	WAL + checkpoints	survive crashes cheaply (Ch. 6)
Execution	parser → planner → executor	turn SQL into rows (Ch. 8)
Maintenance	VACUUM + autovacuum	reclaim dead tuples, freeze xids (Ch. 9)

Return to Figure 1 one last time. Every box in it now has a chapter behind it — and the next time a plan goes wrong or a table will not stop growing, you will know exactly which layer to interrogate, and which command will answer.

References

All structures, parameters, and commands in this book were verified against the official PostgreSQL 18 documentation, listed below in APA style. Because the documentation is continuously updated, each entry notes the retrieval date.

PostgreSQL Global Development Group. (n.d.). *PostgreSQL 18 documentation: Database physical storage*. Retrieved June 2026, from <https://www.postgresql.org/docs/18/storage.html>

PostgreSQL Global Development Group. (n.d.). *PostgreSQL 18 documentation: Reliability and the write-ahead log*. Retrieved June 2026, from <https://www.postgresql.org/docs/18/wal.html>

PostgreSQL Global Development Group. (n.d.). *PostgreSQL 18 documentation: Concurrency control (MVCC)*. Retrieved June 2026, from <https://www.postgresql.org/docs/18/mvcc.html>

PostgreSQL Global Development Group. (n.d.). *PostgreSQL 18 documentation: Routine vacuuming*. Retrieved June 2026, from <https://www.postgresql.org/docs/18/routine-vacuuming.html>



PostgreSQL Global Development Group. (n.d.). *PostgreSQL 18 documentation: Server configuration — resource consumption*. Retrieved June 2026, from <https://www.postgresql.org/docs/18/runtime-config-resource.html>

PostgreSQL Global Development Group. (2025). *PostgreSQL 18 release notes*. Retrieved June 2026, from <https://www.postgresql.org/docs/18/release-18.html>

About TheDBAdmin



TheDBAdmin produces practitioner-grade training for database and reliability engineers: PostgreSQL and MySQL administration, internals, high availability, observability, DevOps, and agentic AI. The Practitioner Series pairs first-principles teaching with hands-on labs whose every command is verified against authoritative sources — so what you learn in the book is what works on the server.

Corporate training, lab books, and course design are available on request.

www.thedbadmin.com • admin@thedbadmin.com • +91 6232090055